
TECHNICAL DOCUMENTATION PACK · DOCUMENT 03

DAAB

Infrastructure, Security & Operations



VERSION
1.0

DATE
2026-09-24

CODE BASELINE
go @ 16dc548 · python @ 6966d4b · web @
907bc8b

Contents

1 Deployment topology

Self-hosted Docker Compose stack · AWS cloud dev environment · Deployment status

2 Build, release & CI/CD

Container images · Release bundle · Database migrations · Pipelines

3 Configuration

Configuration layers · Environment variables by category

4 Scalability

Stateless tiers · Worker queue model · Database & vector indexes · Current constraints

5 Security architecture

Trust boundaries · Authentication · Authorization · Credential encryption · Safe query design · Controlled writes · Audit · Controls summary

6 Data protection & AI data flow

What stays local · What reaches external models · Provider configuration

7 Operations

Logging & metrics · Health endpoints · Backups · Background & recovery jobs

8 Hardening items before production

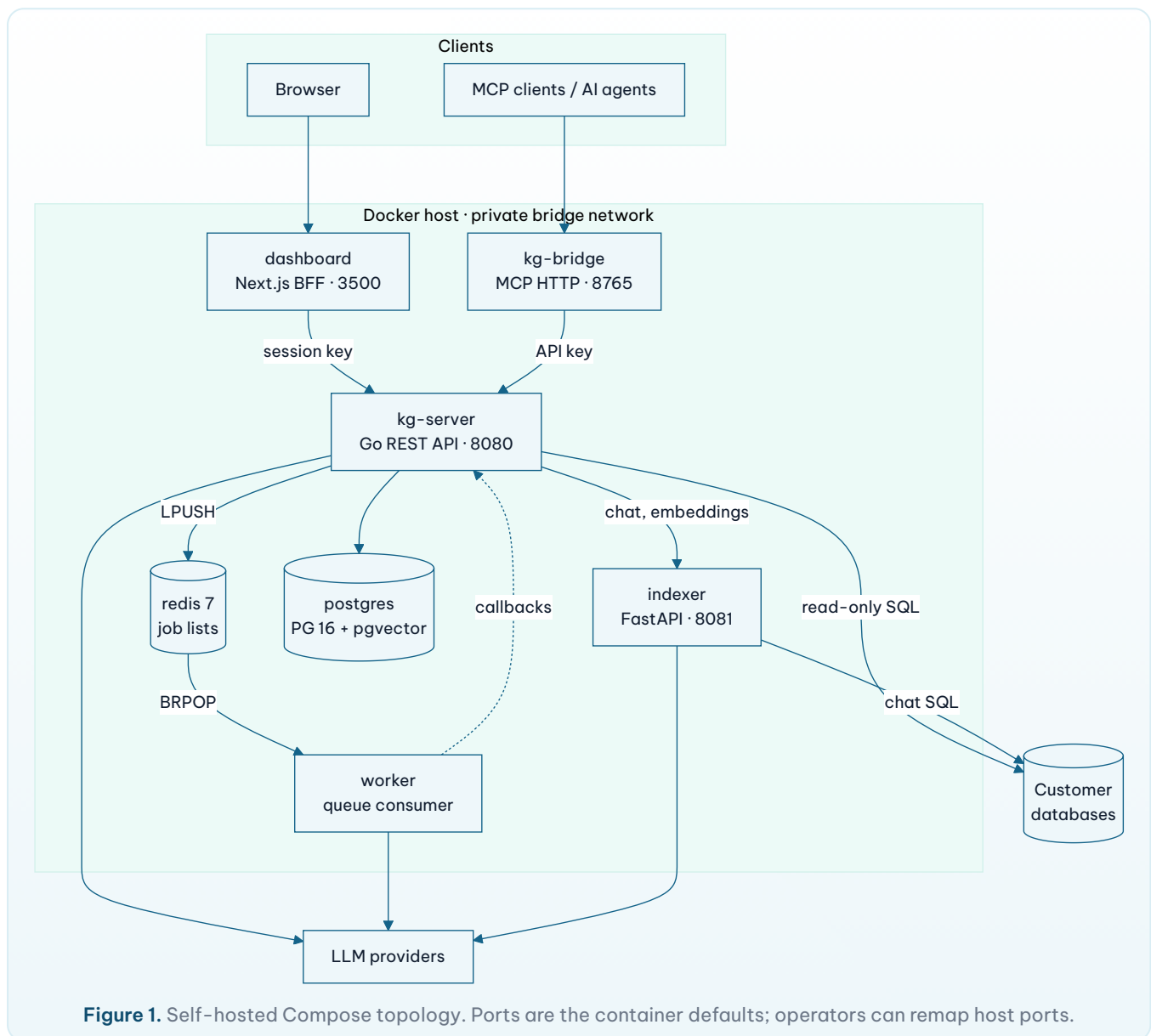
Status vocabulary used throughout: **LIVE** implemented, wired and exercised with real data · **BUILT** implemented and wired, not yet exercised with production data · **PLANNED** roadmap, flag-off or stub.

1 Deployment topology

DAAB is **pre-production**. It runs today as a self-hosted Docker Compose stack for development and demonstrations, using either a source-built development profile or a pinned-image release bundle (v1.0.0). A separate cloud dev environment on AWS runs the platform on managed services (ECS, RDS PostgreSQL, ElastiCache Redis, ALB), and CI/CD builds and deploys the Go service to it.

1.1 Self-hosted Docker Compose stack

The platform runs on a single Docker host with one private bridge network and seven services: PostgreSQL 16 with pgvector, Redis 7, the Go API server, the Python indexer and worker, the Next.js dashboard, and the MCP bridge (the Go image with a different entry point).



Service	Role	State
postgres	System of record: graph, versions, embeddings, users, keys, audit. PostgreSQL 16 + pgvector with data checksums.	Volume postgres_data
redis	Job lists (Go → Python) and short-lived chat clarification state.	Ephemeral
kg-server	REST API (251 routes), NL-query pipeline, source-DB write path, background workers.	Upload volume
indexer / worker	One Python image, two roles: chat streaming and the local embedding service; document ingestion, OCR, extraction, entity resolution, memory embedding.	Upload volume, model cache
dashboard	Web UI plus a backend-for-frontend (BFF) proxy to the API.	Stateless
kg-bridge	MCP server for AI agents over Streamable HTTP (Go image, alternative entry point).	Stateless

Two Compose profiles exist. The **development profile** builds from source, with hot reload for the Go server and bind-mounted repositories for code indexing. The **release profile** (`docker-compose.release.yml`) pulls three pinned images with no bind mounts. It sets `ENVIRONMENT=selfhosted` , a production-like configuration with JSON logs, no verbose errors, no profiling endpoints, secrets supplied only from the environment, and automatic migration on start-up. Its required secrets are declared mandatory, so the stack refuses to start without them.

■ 1.2 AWS cloud dev environment

The repository's Terraform (`deploy/terraform`) defines modules for the VPC, security groups, RDS, ECS (with load balancer, image registry, autoscaling and alarms), secrets and a configuration bucket, with `staging` and `production` profiles. The ECS module defines one service, the Go API server. The managed Redis (ElastiCache) instance and the Python and dashboard tasks used by the cloud dev environment are **not** defined in the repository's Terraform. They are shown dashed in Figure 2.

Network and compute (Terraform)

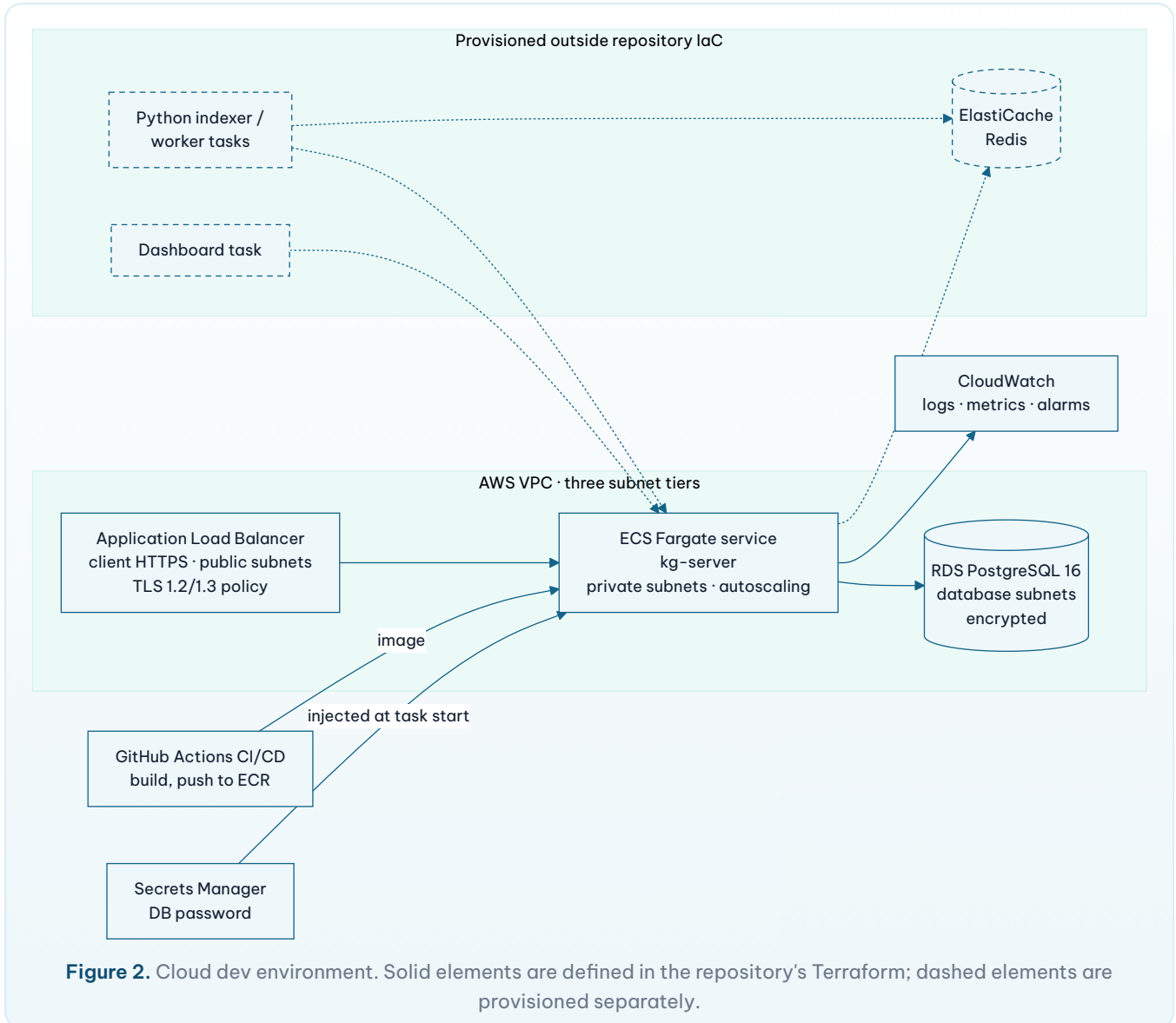
- VPC with public, private and database subnet tiers, and a NAT gateway for outbound traffic from the private tier.
- Security groups allow the chain ALB → ECS → RDS only. RDS accepts traffic from the ECS security group and an optional bastion.
- The ALB has an HTTPS listener with the `ELBSecurityPolicy-TLS13-1-2-2021-06` policy, and HTTP is redirected to HTTPS when a certificate is configured. The health check calls `/healthz` .
- ECS runs on Fargate with no public IPs. Deployments use a circuit breaker with automatic rollback. The staging profile can run on Fargate Spot.

Data, secrets and telemetry (Terraform)

- RDS PostgreSQL 16 has encrypted storage and no public access. Slow-query logging, a statement timeout and `pg_stat_statements` are set through the parameter group.
- The database password is generated into Secrets Manager and injected into the task. SSM parameters for runtime tuning and feature flags are provisioned, but the application does not yet read them (its cloud secrets provider is a stub).
- The configuration bucket on S3 is versioned, encrypted and blocks public access. The deploy workflow publishes configuration files to it; the application does not yet read configuration from S3.
- The credential-encryption key (`KG_ENCRYPTION_KEY`) is not supplied by the Terraform or the deploy workflows and is

provided separately. Without it, the server starts with the encrypted-credential features disabled (Chapter 3).

- CloudWatch receives the log group and alarms on CPU, unhealthy hosts, HTTP 5xx, application error rate and p99 latency.



1.3 Deployment status

Deployment path	Status	Notes
Self-hosted Compose, development profile	LIVE	Used daily for development and demonstrations.
Self-hosted Compose, release bundle v1.0.0	BUILT	Pinned images, selfhosted profile. The release smoke test completed a clean install. The schema has gained migrations since that release.

Deployment path	Status	Notes
AWS cloud dev environment	BUILT	Managed data tier. CI/CD deploys the Go service automatically; the Python and dashboard deployments are not automated.
Production environment	PLANNED	The Terraform <code>production</code> profile exists. There are no production customers, SLAs or measured scale figures.

2 Build, release & CI/CD

2.1 Container images

Image	Build	Size	Runtime user
daab-server	Multi-stage: <code>golang</code> builder → <code>alpine</code> runtime. Static binaries built with <code>CGO_ENABLED=0</code> (<code>kg-server</code> , <code>kg-bridge</code>), with version and commit stamped at link time. The image includes <code>config/</code> and <code>db/migrations/</code> . A separate <code>development</code> stage provides hot reload.	≈ 38 MB	Non-root
daab-python	Two-stage <code>python:3.12-slim</code> with <code>uv sync --frozen --no-dev</code> . It pins CPU-only PyTorch wheels and includes Tesseract OCR with Vietnamese language data. One image runs both the indexer and the worker; its size comes from the local ML runtime (Chapter 6).	≈ 1.7 GB	Non-root
daab-dashboard	Next.js <code>standalone</code> output on <code>node:20-alpine</code> . Public identity-provider settings are inlined at build time.	≈ 284 MB	Non-root

2.2 Release bundle

Releases are produced as a **pinned-image bundle**: three images tagged with commit SHA, semantic version and `latest` , plus `docker-compose.release.yml` . A local release script builds and tags the images and can push them to a private registry. The Compose file takes an optional registry prefix and release tag, so the same bundle can be installed from any registry.

2.3 Database migrations

- **86 versioned SQL migrations**, each an up/down pair managed by `golang-migrate`, create 54 application tables. They cover the PostgreSQL extensions (`uuid-oss` , `pg_trgm` , `vector` , `unaccent`), the knowledge graph, access control, data-source metadata, ingestion, entity resolution and agent memory.
- Migrations are version-tracked and idempotent. The `selfhosted` and development profiles migrate automatically on start-up, and a restart is a no-op.
- A dedicated `kg-migrate` CLI supports `up` , `down` , `version` , `force` and `steps` . It is used for operator-controlled changes and runs from source (Makefile); it is not included in the container images.
- Automatic migration is disabled in the cloud profiles. The CI pipeline's migration step does not yet perform the migration (see §8), so cloud schema changes are currently applied by an operator.

2.4 Continuous integration and delivery

Pipeline	What it does	Status
Go CI (<code>ci.yml</code>)	On pushes and pull requests to the main branches: <code>golangci-lint</code> and a <code>go mod tidy</code> check; migrations applied to a PostgreSQL service container; the full test suite with the race detector and coverage; one integration test that checks cross-project isolation of agent-memory recall; binary and Docker image builds.	LIVE
Go deploy (staging / production workflows)	Build and push to ECR, render a new ECS task definition, deploy with a stability wait, publish the configuration files to S3 (not yet read by the application, §1.2), then run a post-deploy health check through the load balancer.	BUILT
Config deploy	Pushes configuration changes and forces a new ECS deployment without rebuilding the image.	BUILT
Python and dashboard	Python tests and dashboard lint/type-check run locally; no hosted CI workflow. Images are built by the release script. There is no automated deployment to ECS.	PLANNED

Test assets at the code baseline

Go: 2,753 test functions in 333 test files, including 16 integration-tagged files that run against a real PostgreSQL. Python: 943 collected tests; a unit-test run on 2026-09-24 gave 922 passed and 1 failed (the other 20 collected tests did not run). Dashboard: strict TypeScript, with an ESLint configuration that is not enforced (27 errors and 28 warnings outstanding); there is no automated UI test suite yet. Integration tests skip when no database is reachable, so a local "pass" does not by itself prove that they ran.

3 Configuration

3.1 Configuration layers (Go service)

The Go service resolves configuration in four layers. Each layer overrides the previous one:

1. `config/config.yaml` : the domain model. It defines node-type schemas, the edge whitelist (59 rules), completeness rules, search, AI provider strategy and budgets, extraction cost ceilings, and entity-resolution gates.
2. `config/environments/<env>.yaml` : one file per deployment profile (`development` , `staging` , `production` , `selfhosted`). Each sets server timeouts, the connection pool, logging, queue provider, metrics and feature flags.
3. **Environment variables** prefixed `KG_` . In the cloud, the database password is injected into the task environment from Secrets Manager; the encryption key is supplied separately (§1.2).
4. `system_settings` **table**: runtime overrides that administrators edit in the dashboard, cached in-process.

The Python services use typed `pydantic-settings` with defaults; the dashboard reads a few server-side variables plus build-time public identity-provider settings.

■ 3.2 Environment variables by category

Names only. Values are deployment-specific and are never committed. Environment files are excluded from version control.

Category	Variables
Database	<code>KG_DB_HOST</code> <code>KG_DB_PORT</code> <code>KG_DB_NAME</code> <code>KG_DB_USER</code> <code>KG_DB_PASSWORD</code> <code>KG_DB_SSLMODE</code> · <code>KG_MIGRATIONS_DIR</code>
Queues	<code>KG_QUEUE_PROVIDER</code> <code>KG_REDIS_URL</code> <code>KG_REDIS_QUEUE_NAME</code> · <code>REDIS_URL</code> <code>REDIS_QUEUE_NAME</code> <code>REDIS_KG_GENERATION_QUEUE</code> <code>EXTRACTION_QUEUE_NAME</code> <code>AGENT_CONTEXT_QUEUE_NAME</code>
Secrets & crypto	<code>KG_ENCRYPTION_KEY</code> (AES-256 key) · <code>ENCRYPTION_KEY</code> (same key, Python side) · <code>SESSION_SECRET</code> (dashboard cookie)
Service identity	<code>KG_API_KEY</code> · <code>GO_API_URL</code> <code>GO_API_KEY</code> · <code>KG_SERVER_URL</code> <code>KG_PYTHON_URL</code>
Identity provider	<code>KG_SUPABASE_URL</code> <code>KG_SUPABASE_ADMIN_EMAILS</code> · <code>NEXT_PUBLIC_SUPABASE_URL</code> <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> · <code>KG_OAUTH_CLIENT_ID</code> <code>KG_OAUTH_REDIRECT_URI</code>
MCP bridge	<code>KG_MCP_TOKEN</code> <code>KG_MCP_TOKEN_READONLY</code> <code>KG_MCP_AUTH_PASSTHROUGH</code> <code>KG_MCP_REQUIRE_TLS</code> <code>KG_PROJECT_ID</code>
AI & embeddings	<code>EXTRACTION_LLM_URL</code> <code>EXTRACTION_LLM_KEY</code> <code>EXTRACTION_LLM_MODEL</code> · <code>ANTHROPIC_API_KEY</code> · <code>KG_EMBEDDING_API_URL</code> <code>KG_EMBEDDING_API_KEY</code> · <code>EMBEDDING_MODEL_NAME</code> <code>EMBEDDING_SERVICE_URL</code> · <code>KG_PLAN_CACHE</code> <code>KG_VALUE_HINTS</code>
Runtime tuning	<code>WORKER_CONCURRENCY</code> <code>NL_QUERY_TIMEOUT</code> <code>STREAM_TIMEOUT</code> <code>RESOLUTION_*</code> <code>KG_UPLOAD_DIR</code> <code>HF_HOME</code>
Observability	<code>KG_LOG_LEVEL</code> <code>LOG_LEVEL</code> <code>KG_CLOUDWATCH_*</code> <code>KG_METRICS_*</code> <code>KG_VERBOSE_ERRORS</code>
Environment	<code>ENVIRONMENT</code> · <code>KG_CONFIG_DIR</code> <code>KG_APP_CONFIG</code> · <code>NODE_ENV</code> <code>COOKIE_SECURE</code> · <code>KG_AUTH_NOOP</code> (development only)

Fail-safe feature gating

The server starts without `KG_ENCRYPTION_KEY`, but with data-source connectivity, the source-write path, AI provider selection, natural-language query and account linking all disabled. It never falls back to storing credentials unencrypted. The development-only authentication bypass (`KG_AUTH_NOOP`) is honoured only when the environment is `development`.

4 Scalability

The request-serving tiers are stateless and can scale horizontally behind a load balancer. PostgreSQL is the single stateful core. Asynchronous work goes through Redis list queues to a separately scalable worker tier. No throughput or concurrency figures have been measured under production load, and none are claimed here.

■ 4.1 What scales horizontally

- **kg-server (Go), stateless.** All state is in PostgreSQL. Authentication is a per-request hashed key lookup, with no server-side sessions. The Terraform ECS profile autoscales on CPU, memory and requests per target (staging 1–3 tasks; production profile 2–10).
- **kg-bridge, dashboard BFF and Python indexer, stateless.** The bridge builds a per-session MCP server and carries credentials on every request. The dashboard session is an encrypted cookie held

by the browser. The indexer loads its embedding model once per process.

- **Python worker, stateless.** Several workers can consume the same Redis lists. `BRPOP` hands each message to exactly one consumer.
- **PostgreSQL, stateful.** Scales vertically. The RDS profile supports storage autoscaling and Multi-AZ. Logical replication is enabled in the parameter group for future read-replica or change-data-capture use.

■ 4.2 Worker queue model

The Go server publishes JSON job messages to four Redis lists with `LPUSH`, and the Python worker consumes them with a blocking `BRPOP` in first-in, first-out order. The four lists carry code-repository indexing (`enram:kg:indexing`); document drafts, uploaded-file extraction and connector synchronisation (`enram:kg_generation`); chunk-level entity extraction and resolution (`enram:extraction`, the only list with configurable parallel consumers); and agent-memory embedding (`enram:agent_context_embed`).

Long-running synchronous operations are isolated from the request path. Schema synchronisation runs on an in-process job engine with 3 workers, 3 retries and a 30 s backoff. Natural-language queries are submitted asynchronously and long-pollled. Chat responses stream over Server-Sent Events, with at most 3 concurrent chat streams per user.

■ 4.3 Database and vector indexes

- **Graph storage** uses a single node table with a type discriminator and `JSONB` properties. Properties have a GIN index, titles have a trigram GIN index, and there is a weighted full-text `tsvector`. Every update appends a history row in a separate table. Graph traversals are bounded recursive CTEs ($\text{depth} \leq 3$).
- **Vector search** uses three pgvector columns:
 - node and chunk embeddings, `vector(384)` with an HNSW cosine index;
 - agent-memory embeddings, `vector(384)` with an HNSW index;
 - table-description embeddings, `vector(1536)` with an IVFFlat index.

Hybrid search runs the lexical and vector arms concurrently and fuses the results with Reciprocal Rank Fusion. There is no separate vector database to operate, secure or back up.

- **Connection pooling** is configured per profile (for example 25 open and 10 idle connections in `selfhosted`). Queries against customer databases use a fresh, single-connection pool per query, so a slow customer database cannot exhaust the platform's own pool.

■ 4.4 Current constraints (roadmap)

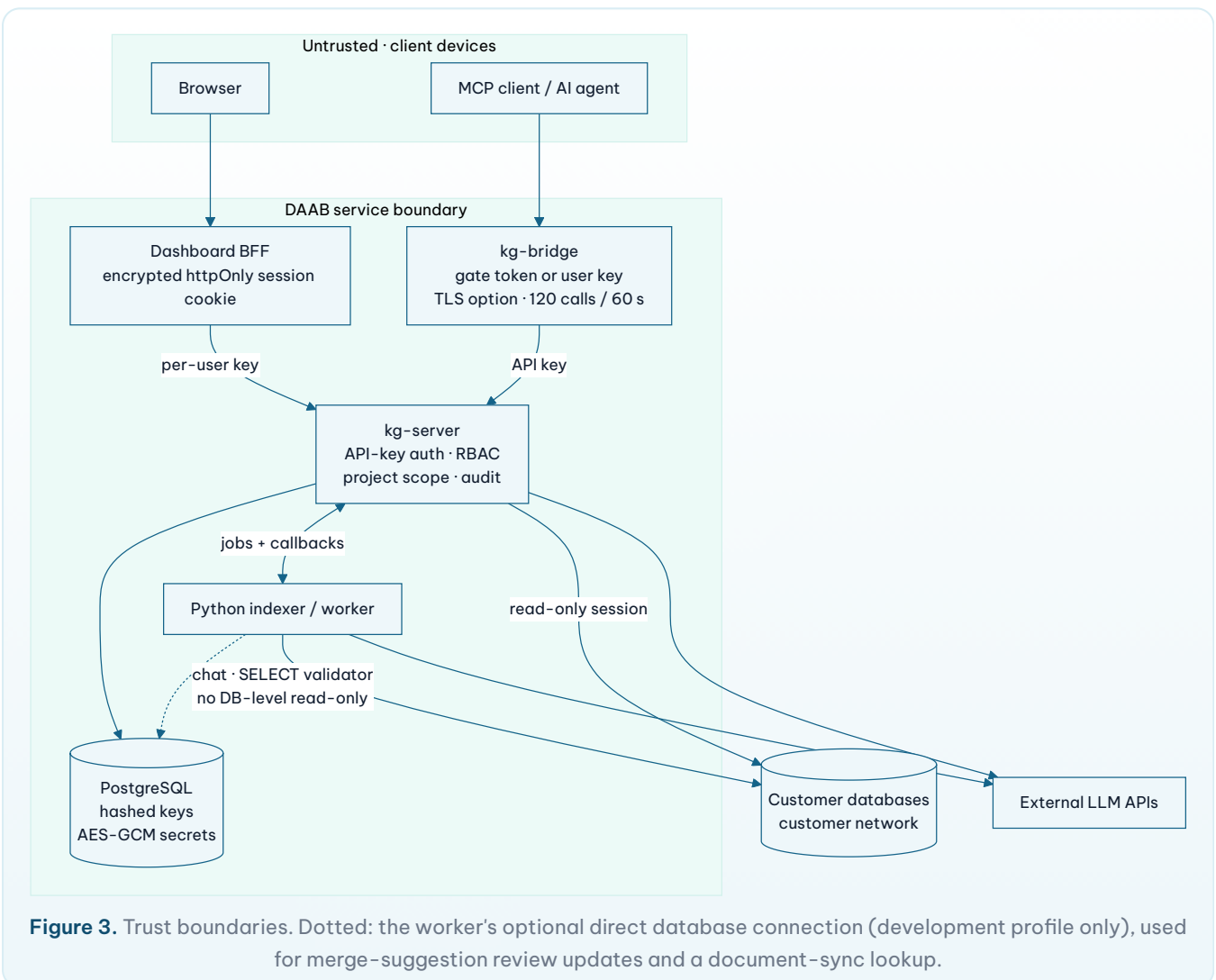
Constraint	Roadmap item
Worker concurrency is cooperative: one process with several asynchronous consumers. CPU-bound local embedding blocks the event loop while it runs.	Move embedding to a thread or process pool, or to the indexer's embedding service. Scale by adding worker replicas rather than raising in-process concurrency.
Queue delivery is at-most-once. <code>BRPOP</code> removes the message, and there is no acknowledgement, visibility timeout or dead-letter queue for worker jobs. A worker crash mid-job loses that job. Per-item failures within a job are isolated and reported.	Adopt reliable-queue semantics (for example Redis Streams consumer groups or SQS): acknowledgement, retry counters, a dead-letter queue, and a scheduled sweep that re-publishes stalled extraction runs. The sweep function exists but is not yet scheduled.

Constraint	Roadmap item
MCP bridge rate limiting is in-process, so the effective limit multiplies with the number of replicas.	Shared (Redis-backed) rate-limit state.
Background workers (chunk linking, memory retention, heartbeat monitor) start in every API replica.	Leader election or a dedicated scheduler task before running multiple API replicas.

5 Security architecture

Requests reach platform data through three entry points: the dashboard BFF, the MCP bridge, and the REST API called directly with an API key. All three converge on the Go API, which authorizes access to platform data; the realtime progress routes are the exception (§8, item 7). Customer database credentials are encrypted at rest. On the API and MCP query path, AI models never execute SQL directly: they produce a structured plan that deterministic code turns into parameterized, read-only SQL. Dashboard chat is handled differently: the Python indexer connects to the customer database itself (§5.5).

5.1 Trust boundaries



■ 5.2 Authentication

- **API keys** **LIVE**. Keys come from a cryptographic random source, and only a SHA-256 hash and a short display prefix are stored; the plaintext is shown once. Each key has a role, an optional project scope, revocation and last-use tracking. Every `/api/v1` route requires a key except the two login endpoints.
- **Password login** **LIVE**. Passwords are bcrypt hashes, and unknown users take a timing-equalised path. Accounts lock after 5 failed attempts, with administrator unlock, and a password change can be forced. Each password login mints a **per-user session key** and revokes the previous one, which enforces a single active dashboard session.
- **Dashboard session** **LIVE**. An `iron-session` encrypted cookie (`httpOnly`, `SameSite=lax`, `secure` in production) carries the user's key only inside the encrypted cookie, so browser script never sees an API key. When the key is revoked upstream, the cookie is destroyed.
- **Single sign-on** **BUILT**. Optional. Identity-provider JWTs are verified against a cached JWKS, with issuer and audience checks. Users are provisioned just in time as viewers; administrators are set by configuration.
- **MCP bridge** **LIVE**. Two modes:
 - *gate-token*: a shared token that must be at least 32 bytes long (decoded hex or base64url) or 43 characters, checked at start-up. An optional second token grants a read-only scope.
 - *passthrough* (multi-tenant): each caller presents its own DAAB API key, which is forwarded per request, so the API enforces that caller's role and projects.

The bridge does not terminate TLS. It warns on non-loopback binds, and refuses to start without TLS when `KG_MCP_REQUIRE_TLS=true`.

- **AI account linking** **BUILT**. OAuth with PKCE. Tokens are encrypted at rest and refreshed by a background worker.

■ 5.3 Authorization

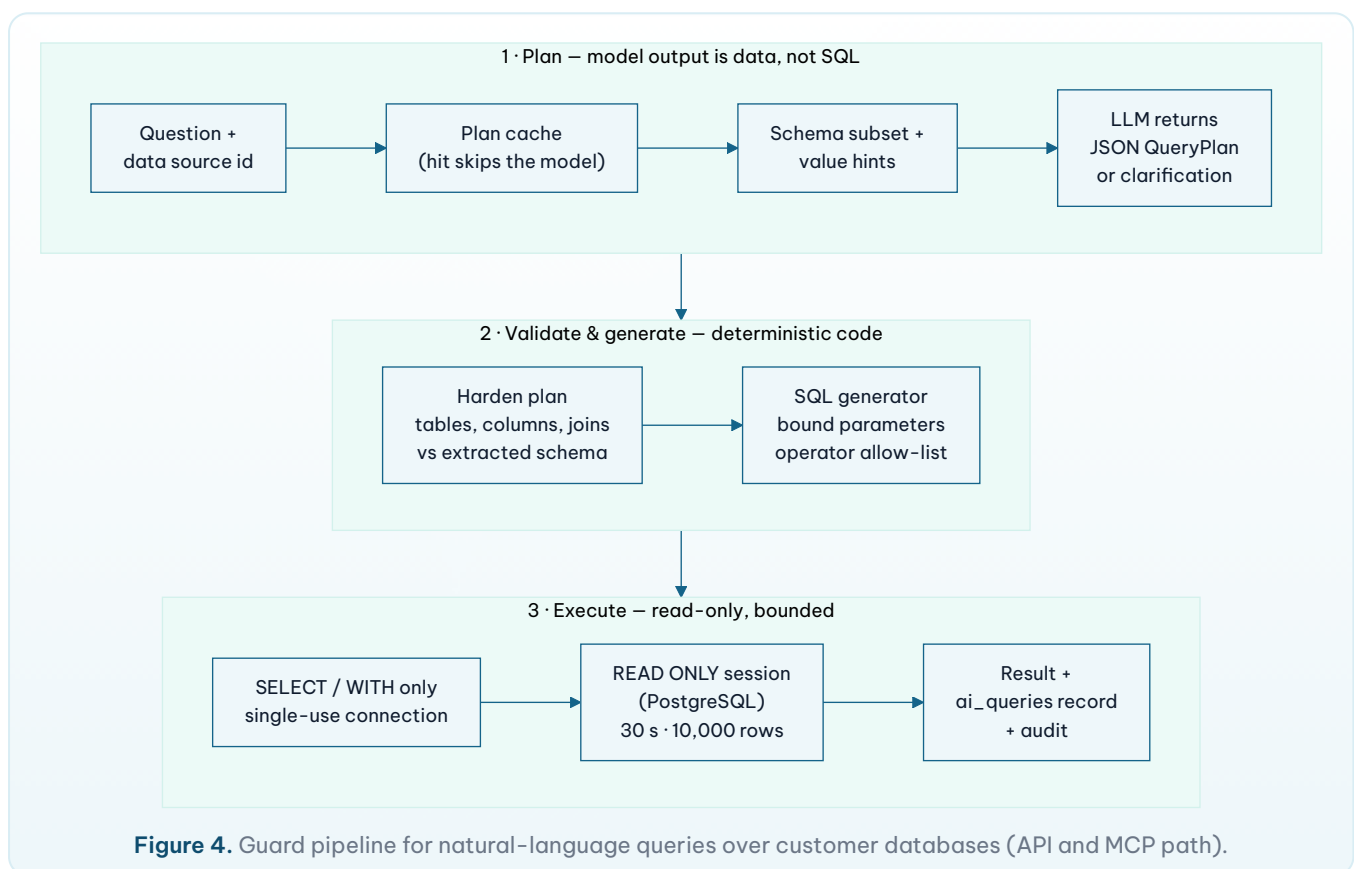
- **Roles**. Keys carry `admin`, `developer`, `agent` or `viewer`. Project membership carries `admin`, `developer` or `viewer`, with an additive hierarchy.
- **Project resolution middleware** resolves the effective project for every `/api/` request from the key's default, its project list or the user's memberships. It enforces an explicit override permission and blocks all mutations on archived projects.
- **Handler-level checks** re-verify project access for identifiers passed in request bodies and on project-scoped routes; several by-ID read routes (node history, document outline and backlinks) also verify the resource's project. Writes to nodes, edges, data sources, source writes and connectors require a minimum project role for user-linked keys.
- **MCP read-only scope** is enforced per tool at dispatch. Tools are classified read or write, and write tools carry no read-only hint, so compliant MCP clients show a confirmation prompt before calling them.
- Applying a per-project check uniformly on every by-ID read, and to service and agent keys that are not linked to a user, is a hardening item (Chapter 8, item 4).
- **Project pinning** in the bridge overrides model-supplied project identifiers. It protects against model mistakes but is not the tenancy boundary; tenancy is enforced by the API key's scope.

■ 5.4 Credential encryption

Customer database connection strings, AI provider API keys, OAuth tokens and connector credentials are encrypted with **AES-256-GCM**. The server uses a 32-byte key supplied at runtime, generates a fresh random 12-byte nonce for every encryption, and stores `nonce || ciphertext || tag`, so any tampering is detected. Decryption happens only in memory, at the moment a connection is opened. The dashboard's chat mode forwards the *still-encrypted* connection string to the Python service, which holds the same key. For dashboard chat, the server also passes the selected AI provider's key to the Python service in a request header over the internal network, and the Python service calls the provider directly. Consolidating key custody and authenticating internal service calls are listed in §8.

■ 5.5 Safe query design

The natural-language query pipeline behind the API and MCP tool `kg_query_datasource` is built so that the model never writes executable SQL:



- **Plan, not SQL.** The model emits a typed `QueryPlan` (tables, joins, filters, aggregations, grouping, ordering, limit). All values become bound parameters. Operators must be on an allow-list, and the few permitted expression shapes (intervals, scalar sub-queries) must match closed grammars.
- **Schema grounding.** Plans are repaired and validated against the *extracted* schema: column existence, join promotion and de-duplication, ON-clause validity, and a cap of 100 on ranked queries. If execution fails, the pipeline makes one simplification retry.
- **Execution guards.** The generated statement must start with `SELECT` or `WITH`. Each query uses a fresh single-connection pool. On PostgreSQL sources the session is additionally set to `READ ONLY` at the database level. MySQL, MariaDB and SQL Server rely on the deterministic generator and the statement-prefix check. A 30 s timeout and a 10,000-row hard cap (with a `truncated` flag) apply to all dialects.

- **Least privilege at source.** Customers are advised to register read-only database credentials, the strongest guard and independent of DAAB.

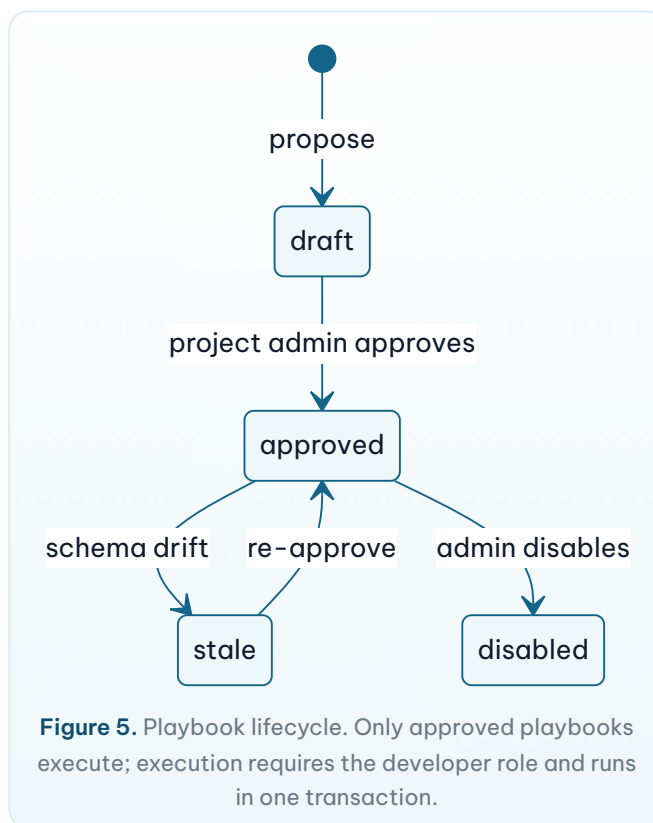
Dashboard chat has two further modes, implemented in the Python indexer, which opens its own connection to the customer database (Figure 3). The *fast* mode uses the same plan-then-generate approach with bound values. The *agentic* mode lets the model compose `SELECT` statements inside a tool loop. These are checked by a statement validator (single statement, `SELECT` / `WITH` only, a blocklist of data-changing keywords, a length limit) and results are trimmed to 50 rows after execution and 20 rows per tool result. Neither Python mode sets a database-level read-only session. Tightening both is listed in §8.

■ 5.6 Controlled writes

Writes to customer databases are **off by default** and pass four independent controls:

1. **Per-source opt-in.** `allow_writes` is false until an administrator enables it.
2. **Table whitelist.** Only listed tables are writable. An empty list means nothing is writable.
3. **Validation** against extracted metadata: required and unknown columns, types and maximum lengths.
4. **Audit.** Every attempt, whether successful, rejected or failed, is written to `source_write_audit`.

Multi-step writes are expressed as named, versioned **playbooks**. Playbooks are declarative server-side step definitions, proposed deterministically (no model involved) and approved by a project administrator. They execute in a **single database transaction**. After every schema sync, playbooks whose target schema has drifted are marked *stale* automatically. The write path currently supports PostgreSQL sources.



■ 5.7 Audit logging

Security-relevant activity is recorded in dedicated tables: `audit_trail` (node creation, settings changes, schema syncs, query submissions); `kg_audit_log` (reads of conversation history); `source_write_audit` (every write attempt against a customer database); `knowledge_node_versions` (version history written by a database trigger on every update; graph edits only append versions, while regenerating a document replaces its sections and chunks together with their version rows); and `ai_usage_logs` with budget tables (per-request model usage and cost attribution). Administrators query the audit log and per-entity history through the API.

■ 5.8 Controls summary

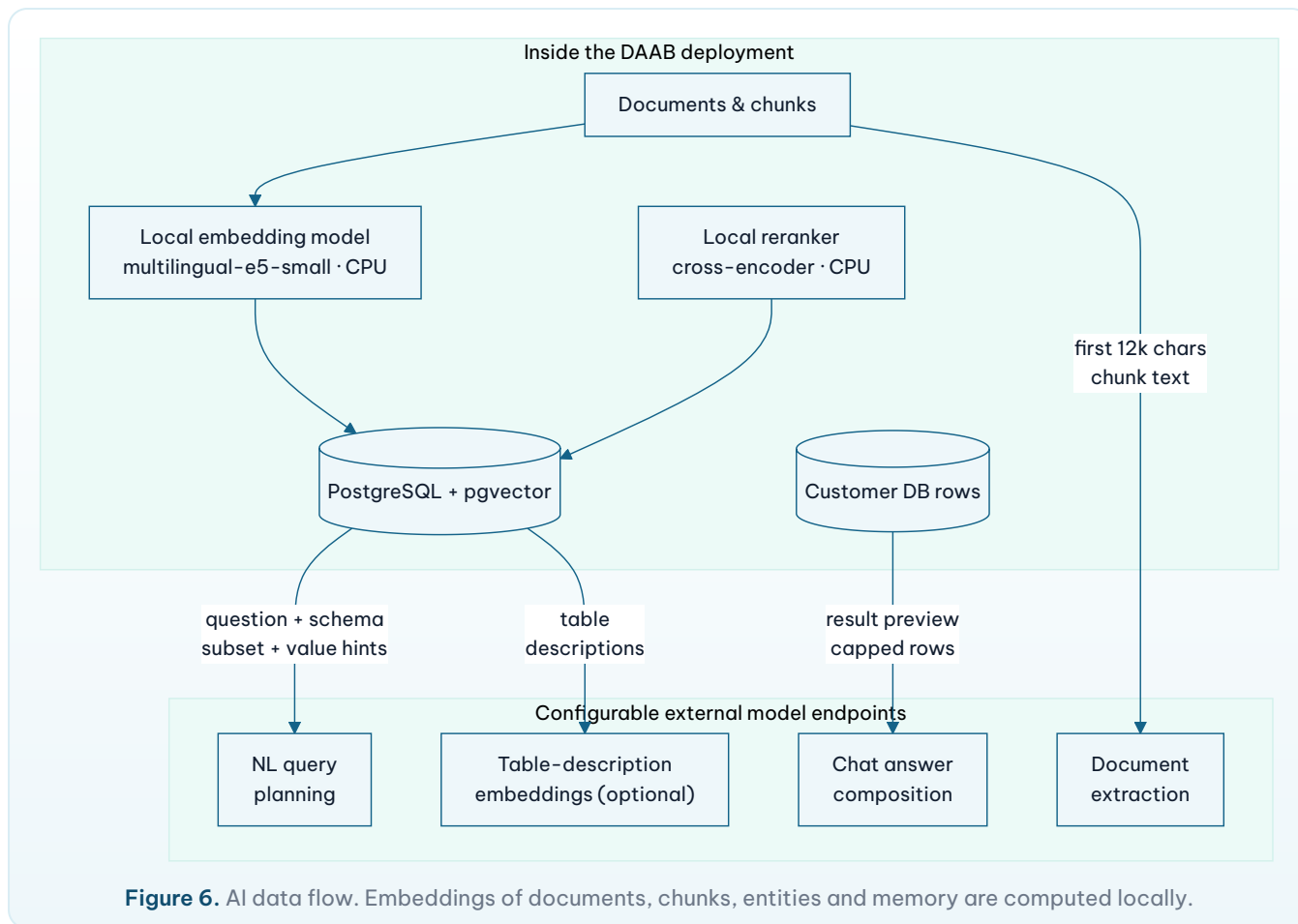
Control	Implementation	Status
Key storage	SHA-256 hashed API keys; bcrypt passwords	LIVE
Secret encryption at rest	AES-256-GCM for DSNs, provider keys, OAuth and connector credentials	LIVE
Session security	Encrypted <code>httpOnly</code> cookie; single active dashboard session; forced password change	LIVE
Brute-force protection	Login lock-out after 5 failures	LIVE
Project isolation	Project middleware, project-scoped handler checks, body-level re-checks; one CI isolation test (agent-memory recall). Uniform by-ID checks: §8, item 4	LIVE
Role-based access	Key roles plus per-project membership roles on write paths	LIVE
MCP read-only scope	Separate token; write tools refused at dispatch	BUILT
MCP rate limit	120 calls per 60 s per identity (in-process)	LIVE
API-wide rate limit	Not yet enforced (§8)	PLANNED
Query safety	API and MCP path: plan-based SQL generation, bound parameters, read-only session (PostgreSQL), timeout, row cap	LIVE
Controlled writes	Opt-in, whitelist, validation, audit, approved single-transaction playbooks (small volume)	LIVE
Transport security	ALB TLS (cloud); optional TLS enforcement on the bridge	BUILT
Container hardening	Non-root users, static Go binaries, minimal base images	LIVE
Error hygiene	Verbose errors, request-body logging and profiling endpoints disabled in production-like profiles	LIVE

6 Data protection & AI data flow

DAAB computes embeddings and reranking on local CPU models inside the deployment, so document content is never sent to a third party for embedding. External language models are called only for specific steps, and each step sees a bounded subset of data. All providers are configurable, including self-hosted OpenAI-compatible endpoints.

■ 6.1 What stays local

Embeddings: document sections, chunks, entities and agent-memory records are embedded with `intfloat/multilingual-e5-small` (384 dimensions) on CPU in the Python service, and search queries are embedded by the same local service. **Reranking:** entity-resolution candidates are filtered by a local cross-encoder. **OCR and parsing:** Tesseract (including Vietnamese) and local layout OCR run in the worker, and temporary files from connector syncs are deleted after processing. **Query execution:** SQL runs from the DAAB deployment to the customer database, and results are stored only in DAAB's own database and returned to the caller.



■ 6.2 What reaches external models

Step	Data sent to the model	Trigger
Document extraction	The first 12,000 characters of each new or changed document (topic, entities, referenced tables). Content-hash deduplication means a file is never sent twice.	Every ingest
Chunk-level entity extraction	Full chunk text (about 1,200 characters per chunk, up to about 1,500 for a single-chunk section) for typed entities and relations.	Explicit request, with cost ceilings
Entity-resolution verification	Candidate entity names and short descriptions, in small batches.	After extraction
NL query planning	The question, the relevant subset of the extracted schema (tables, columns, types, keys) and a few sampled categorical values used as spelling hints (configurable).	Each query not answered from the plan cache
Chat answer composition	The question and a result preview (about 2,000 characters in fast mode; up to 20 rows per tool result in agentic mode) for summaries, formatting and insights.	Dashboard chat
Table-description embeddings	Schema table descriptions, used to select tables in very large schemas.	Optional, only when configured

When an AI agent calls DAAB through MCP, the results it receives are processed by **the agent's own model**, outside DAAB's boundary. Which data goes on to that model is governed by the scope of the key the agent holds.

■ 6.3 Provider configuration

- **Provider registry.** AI providers are registered in the database with encrypted credentials. The server routes each request through a priority-ordered selector with failover and a circuit breaker. Models can be assigned per function, such as planning, extraction or verification.
- **Supported interfaces:** the Anthropic API, a linked Claude subscription (OAuth), and any **OpenAI-compatible** endpoint. The second option allows a customer to point DAAB at a regional provider or a self-hosted model server, so that no data leaves their environment.
- **Cost controls.** Extraction runs are estimated before they start and refused above per-run and per-document ceilings. Every request proxied through the server's AI layer carries a request type and run identifier for cost attribution. Two paths call providers directly and are not recorded in that usage log or covered by its circuit breaker: an optional direct extraction endpoint (`EXTRACTION_LLM_URL`) used for chunk-level extraction, verification and adjudication, and dashboard chat in the Python service. Plan caching and an empty-result fast path avoid unnecessary calls.
- **Data-processing terms** with each external provider depend on the customer's deployment choice:

TO BE COMPLETED

7 Operations

■ 7.1 Logging and metrics

- **Structured logs.** The Go service uses `log/slog` (JSON in production-like profiles) with a request-ID middleware, access logging and panic recovery. Request bodies are not logged in those profiles.
- **CloudWatch (optional).** Logs can be shipped to CloudWatch Logs in batches with a configurable retention, and an in-process metrics collector can publish to CloudWatch Metrics every 60 s. The self-hosted profile writes to standard output only, and ECS collects container output into its log group.
- **Alarms (Terraform).** CPU, unhealthy targets, HTTP 5xx, application error rate and p99 latency.
- **Python services** log through the standard library at a configurable level. Per-item failures in the ingestion pipeline are logged at warning level or above, never swallowed silently.

■ 7.2 Health endpoints

kg-server, kg-bridge and the Python indexer each expose `GET /healthz` (liveness) and `GET /readyz` (readiness), outside authentication. Readiness checks the dependencies each service needs: the database ping for kg-server (503 when unavailable), the API for the bridge, and the API and Redis for the indexer. The indexer reports a failed dependency as `degraded` in the response body but still returns HTTP 200, so orchestrators cannot yet use it to hold back traffic. PostgreSQL and Redis use Compose health checks (`pg_isready` , `redis-cli ping`), and application services wait until these are healthy. The ALB target group uses `/healthz` .

■ 7.3 Backups

Environment	What exists	Status
Cloud (RDS, Terraform profile)	Automated RDS backups with a defined window (3-day retention in the staging profile, 14 days in the production profile), encrypted storage, snapshot tag propagation, a final snapshot and deletion protection in production, and Multi-AZ in production.	BUILT
Configuration (S3)	Versioned, encrypted configuration bucket with lifecycle retention of prior versions.	BUILT
Self-hosted Compose	Data lives in named Docker volumes (database and uploads). The bundle includes no scheduled backup, retention or restore procedure.	TO BE COMPLETED
Restore testing / RPO & RTO targets	Not yet defined.	TO BE COMPLETED

Chunks, embeddings and schema metadata can be rebuilt from uploads and customer databases; the irreplaceable state is the PostgreSQL database and the upload volume.

■ 7.4 Background and recovery jobs

Job	Purpose	Status
Sync-job heartbeat monitor	Checks every 60 s and fails schema-sync jobs that have shown no progress for 30 minutes, so the UI never shows a job as running forever.	LIVE
Chunk-link worker	Every 5 minutes, builds cross-document similarity edges for newly ingested content.	LIVE
Memory retention worker	Deduplicates and bounds the growth of agent memory, and records recall timestamps.	BUILT
OAuth token refresh	Refreshes linked AI-account tokens before they expire.	BUILT
Stalled-extraction sweep	Re-publishes extraction runs stuck mid-flight. The function is implemented but not yet scheduled.	PLANNED
Graceful shutdown	API and worker handle SIGINT/SIGTERM. The worker removes orphaned temporary files at start-up.	LIVE

8 Hardening items before production

These items are scheduled before any production deployment. None requires an architectural change: each hardens an existing control or completes existing wiring.

#	Item	Scope
1	Secret management via cloud secret store	Implement the in-app cloud secrets provider (currently a stub) so the provisioned SSM parameters and S3 configuration are read, supply the encryption key through infrastructure-as-code, and move it to a managed key service, narrowing custody (today shared by the Go and Python services) to the fewest services.

#	Item	Scope
2	TLS on the MCP bridge by default	Make <code>KG_MCP_REQUIRE_TLS</code> the default for non-loopback binds, and ship a reference TLS reverse-proxy configuration.
3	API-wide rate limiting	Enforce per-key and per-IP limits on the REST API and login endpoints (today only the MCP bridge and the public ingestion endpoint limit calls, per identity and in-process), with shared state across replicas.
4	Service-key scoping and per-project authorization	Apply per-project role and ownership checks uniformly, including for agent and service keys, on every project- and data-source-scoped operation and every by-ID read.
5	Session enforcement on all dashboard API routes	Enforce the session middleware on every dashboard API route.
6	Service-to-service authentication	Validate credentials on the internal Python endpoints (chat, embeddings, admin) rather than relying on network placement.
7	WebSocket and realtime origin checks	Restrict WebSocket origins to configured hosts, and require authentication on the realtime progress routes.
8	Remove development seed credentials from release builds	Move development seed data out of the standard migration chain, and require operator-generated credentials at install time.
9	Network exposure of internal services	Stop publishing database, Redis and indexer ports in the release bundle and enable Redis authentication; expose only the dashboard and bridge, behind TLS.
10	Job acknowledgement and retry	Reliable queue semantics for worker jobs (acknowledgement, retry, dead-letter), and scheduling of the stalled-extraction sweep (§4.4).
11	SQL validation hardening	Database-level read-only enforcement for every supported engine (today PostgreSQL only, API path); parser-based validation, extended identifier checks, read-only sessions and statement timeouts on the dashboard chat (Python) paths; least-privilege credential guidance.
12	Cloud migration step and full-stack delivery	Use the <code>kg-migrate</code> binary in the deploy workflows, add CI and ECS delivery for the Python and dashboard services, and bring managed Redis under infrastructure-as-code.
13	Multi-replica readiness	Coordinate background workers across API replicas, align HTTP timeouts with long-poll and streaming routes, make the indexer readiness check fail when a dependency is down, and define self-hosted backup, restore and RPO/RTO.
14	Security headers and external review	Add a content security policy and standard security headers on the dashboard, then commission an independent penetration test before the first production tenant: TO BE COMPLETED
15	Uniform model-call accounting	Route the direct extraction endpoint and dashboard chat model calls through the same usage log, budgets and circuit breaker as the server's AI layer.

Source references (repository-relative, at the code baseline): `ennam.kg.go/cmd/kg-server/main.go` , `internal/middleware/` , `internal/handler/project_role.go` , `internal/crypto/aes.go` , `internal/service/source_executor.go` , `internal/bridge/` , `deploy/terraform/` , `.github/workflows/` , `docker-compose.release.yml` .